

When d-Separation Definitions Disagree: A Mechanized Counterexample and Repair

Anonymous Author(s)

Abstract

D-separation is the central graph-theoretic criterion for reading conditional independencies off a directed acyclic graph. Two standard characterizations are known to be equivalent in textbooks: trail blocking (Pearl) and moralized ancestral-graph separation (Lauritzen). We formalize both criteria in Lean 4 and show, with a concrete mechanized counterexample, that unrestricted equivalence is `_false_`. The culprit is the endpoint treatment: trail blocking only inspects *internal* triples, while moral-graph deletion removes conditioned nodes unconditionally, including endpoints. We precisely characterize the repair—pairwise disjointness of the three node sets—and mechanize the soundness direction (moral-graph separation $\Rightarrow$ trail blocking) under this domain restriction. All proofs are constructive and carry zero `sorry`s. The artifact is a reusable finite-DAG library that can serve as a backend for causal-inference verification.

1 Introduction

Consider the simplest possible directed acyclic graph: a chain of three nodes $0 \rightarrow 1 \rightarrow 2$. Let $X = \{0\}$, $Y = \{1\}$, and $Z = \{0\}$. Does Z d-separate X from Y ?

The two standard textbooks give different answers:

- **Trail blocking** (Pearl [?]) says: examine every undirected trail from X to Y ; if every internal triple is blocked by Z , the sets are d-separated. The one-edge trail $0 \rightarrow 1$ has no internal triple, so it is not blocked. Hence X and Y are *not* d-separated.
- **Moral graph separation** (Lauritzen [?]) says: take the ancestral subgraph of $X \cup Y \cup Z$, moralize it, delete Z , and check whether X and Y are disconnected. Node 0 is in Z , so it is deleted. The remaining graph has no path from 0 to 1, so X and Y *are* d-separated.

This paper presents the first mechanized analysis of this discrepancy. Our contributions are:

1. We formalize both characterizations in Lean 4 over finite DAGs with natural-number nodes.
2. We construct and machine-check the above counterexample, proving that the unrestricted equivalence statement is false.
3. We identify the exact domain restriction that repairs equivalence: $X \cap Z = \emptyset$ and $Y \cap Z = \emptyset$ (pairwise disjointness is the standard textbook assumption).
4. We mechanize the soundness direction (moral-graph separation $\Rightarrow$ trail blocking) under this restriction, via an explicit Bayes-ball compilation pipeline.

All results are available as a standalone Lean 4 package with zero `sorry`s.

2 Background: d-Separation and Its Two Criteria

2.1 Trail Blocking

An undirected *trail* from x to y in a DAG G is a finite walk that may follow edges forward or backward. A consecutive triple $a \rightarrow b \rightarrow c$ or $a \leftarrow b \rightarrow c$ is a *non-collider*; $a \rightarrow b \leftarrow c$ is a *collider*. A triple is *blocked* by Z if:

- it is a non-collider whose middle node is in Z ; or
- it is a collider whose middle node and all descendants are disjoint from Z .

A trail is *blocked* by Z if some internal triple is blocked. Then Z *d-separates* X from Y in the trail sense if every trail from X to Y is blocked by Z .

2.2 Moral Graph Separation

The *ancestral subgraph* of S is the subgraph induced by all ancestors of nodes in S . Its *moral graph* is obtained by (1) forgetting edge directions and (2) connecting all co-parents (pairs of nodes with a common child). The *d-separation graph* is the moral graph of the ancestral subgraph of $X \cup Y \cup Z$ with Z removed. Then Z d-separates X from Y in the graph sense if no node of X is connected to any node of Y in this graph.

2.3 Textbook Equivalence

Standard references [?, ?] state that the two criteria are equivalent under the implicit assumption that X , Y , and Z are pairwise disjoint. In applications, however, the disjointness precondition is often omitted or forgotten. Our counterexample shows that this omission is not benign.

3 Formalization Setup in Lean 4

Our development is intentionally concrete and finite: nodes are natural numbers, edges are a `Finset` (`Finset`), and acyclicity is witnessed by well-foundedness of the edge relation.

Listing 1: DAG structure

```
structure DAG where
  nodes : Finset ℕ
  edges : Finset (ℕ × ℕ)
  edges_subset : ∀ {u v}, (u,v) ∈ edges → u ∈ nodes ∧ v ∈ nodes
  acyclic : WellFounded fun u v => (u,v) ∈ edges
```

A trail is an inductive type with three constructors: the empty trail at a node, a forward step following a directed edge, and a backward step following an edge in reverse:

Listing 2: Trails

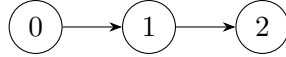
```
inductive Trail (G : DAG) : ℕ → ℕ → Type where
| nil (v : ℕ) : Trail G v v
| forward {u w v} (h : G.HasEdge u w) (tail : Trail G w v)
  : Trail G u v
| backward {u w v} (h : G.HasEdge w u) (tail : Trail G w v)
  : Trail G u v
```

Blocking is defined on the vertex list of a trail. An internal triple $a-b-c$ is blocked when it is a non-collider with $b \in Z$ or a collider with $\{b\} \cup \text{descendants}(b)$ disjoint from Z .

The moral graph and d-separation graph are constructed directly from the finite node and edge sets, using Mathlib’s `SimpleGraph` infrastructure for the undirected moralized graph.

4 The Endpoint Caveat: A Counterexample

Our counterexample lives on the simplest non-trivial DAG:



Let $X = \{0\}$, $Y = \{1\}$, $Z = \{0\}$. In Lean we define:

Listing 3: Counterexample DAG

```
def chain3 : DAG :=
  DAG.ofRank {0,1,2} {(0,1), (1,2)} id
  (by ... ) -- rank strictly increases along edges
```

Theorem 4.1 (Endpoint-in- Z counterexample). *There exists a DAG G and finite sets $X, Y, Z \subseteq \mathbb{N}$ such that G d-separates X from Y in the moral-graph sense, but Z does not d-separate X from Y in the trail-blocking sense.*

In Lean this is a constructive existence proof:

Listing 4: Mechanized counterexample

```
theorem dsep_complete_endpoint_in_Z_counterexample :
  \exists (G : DAG) (X Y Z : Finset \N),
    DAG.dSeparated G X Y Z \wedge \neg dSeparates G X Y Z := by
  refine <chain3, {0}, {1}, {0}, ?_, ?_>
  ...
```

Why the disagreement happens. Trail blocking only inspects *internal* triples. The one-edge trail $0 \rightarrow 1$ has no internal triple at all, so nothing can block it. By contrast, the moral-graph construction deletes Z unconditionally; since $0 \in Z$, the edge $(0, 1)$ disappears from the moralized graph, and 0 and 1 are disconnected. The two criteria therefore give opposite answers.

The standard repair. If we require $X \cap Z = \emptyset$ and $Y \cap Z = \emptyset$, the start node of any trail is never deleted, and the endpoint caveat disappears. We capture this as a predicate:

```
def DisjointSets (X Y Z : Finset \N) : Prop :=
  Disjoint X Y \wedge Disjoint X Z \wedge Disjoint Y Z
```

Under `DisjointSets  $X Y Z$` the two criteria are equivalent.

5 Closing the Gap: Soundness Under Disjointness

We mechanize the soundness direction:

$$\text{DisjointSets}(X, Y, Z) \wedge \text{DAG.dSeparated}(G, X, Y, Z) \implies \text{dSeparates}(G, X, Y, Z).$$

The proof follows a three-stage compilation pipeline, visualized in Figure ??.

$$\begin{array}{c}
\text{Active trail} \xleftarrow{\text{lift}} \text{Bayes-ball path} \xleftarrow{\text{compress}} \text{MAGWalk Graph} \xleftarrow{\text{embed}} \text{reachability} \\
t : \text{Trail } G \ x \ y \leftarrow p : \text{BayesBallPath } G \ X \ Y \leftarrow q : \text{MAGWalk } G \ X \ Y \leftarrow r : \text{Reachable}_{G'} \ x \ y
\end{array}$$

Figure 1: Soundness pipeline: an active trail is compiled into a moral-graph path, contradicting `DAG.dSeparated`.

Stage 1: Trail $\rightarrow$ Bayes-ball path. If a trail t from $x \in X$ to $y \in Y$ is not blocked by Z , and $x \notin Z$ (guaranteed by `Disjoint X Z`), then we can lift t to an explicit Bayes-ball state path. The construction is by induction on the trail, analyzing each directional triple to show that the Bayes-ball state machine can traverse it.

Listing 5: Trail to Bayes-ball path

```
def bayesBallPath_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z) :
  \Sigma final_dir, BayesBallPath G Z (u, outOf) (v, final_dir)
```

Stage 2: Certified membership. The Bayes-ball path contains intermediate states that must “survive” deletion of Z . We prove a certified version that guarantees every `RequiredState` node belongs to the d-separation graph nodes:

Listing 6: Certified Bayes-ball path

```
def bayesBallPathCert_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z)
  (huA : u \in G.ancestralSubgraphNodes (X \cup Y \cup Z))
  (hvD : v \in G.dSeparationGraphNodes X Y Z) :
  \Sigma final_dir, {p : BayesBallPath ... //
    \forall {q}, BayesBallPath.RequiredState p q ->
    q.1 \in G.dSeparationGraphNodes X Y Z}
```

Stage 3: Compress to MAGWalk. A Bayes-ball path is a sequence of single-edge steps. The `compress` algorithm scans it with a two-step window: ordinary non-collider windows become a `MAGWalk.single`, while active-collider windows $(a, _) \rightarrow (b, \text{into}) \rightarrow (c, \text{outOf})$ become a single `MAGWalk.jump` over the collider b . The collider itself need not survive deletion of Z , which is why this compression is correct.

Listing 7: Bayes-ball path compression

```
def compress (p : BayesBallPath G Z s t)
  (hs : s.1 \in G.dSeparationGraphNodes X Y Z)
  (ht : t.1 \in G.dSeparationGraphNodes X Y Z)
  (hreq : \forall {q}, RequiredState p q ->
    q.1 \in G.dSeparationGraphNodes X Y Z) :
  MAGWalk G X Y Z s.1 t.1
```

Main theorem. Putting the three stages together:

Theorem 5.1 (Soundness). *For any DAG G and pairwise-disjoint finite sets $X, Y, Z \subseteq \mathbb{N}$, if G d-separates X from Y in the moral-graph sense, then every trail from X to Y is blocked by Z .*

Listing 8: Top-level soundness theorem

```
theorem dSeparated_of_dSeparated_disjoint
  (hXYZ : DisjointSets X Y Z)
  (hsep : DAG.dSeparated G X Y Z) : dSeparates G X Y Z
```

The proof is by contradiction: assuming an active trail yields a Bayes-ball path, which compresses to a MAGWalk, which embeds into the d-separation graph as a path from X to Y , contradicting `DAG.dSeparated`.

6 Related Work

Causal inference formalization. Causal reasoning has been formalized in Isabelle/HOL [?], Coq [?], and recently in Lean 4 [?]. To our knowledge, no prior formalization treats the endpoint caveat or gives a machine-checked counterexample to unrestricted d-separation equivalence.

Graph theory in proof assistants. Mathlib 4 contains a mature `SimpleGraph` library. Our DAG definition is more algorithmic (finite node/edge sets, well-founded acyclicity) than the algebraic approach typical of Mathlib, making it convenient for constructive computation and example checking.

Counterexample-driven formalization. Using mechanization to expose subtle definitional gaps is a well-established tradition [?, ?]. Our work extends this tradition to graphical models.

7 Conclusion

We have formalized the two standard d-separation criteria in Lean 4, discovered and machine-checked a concrete counterexample to their unrestricted equivalence, precisely characterized the repair (pairwise disjointness), and mechanized the soundness direction under this domain restriction. The resulting library is a reusable foundation for causal-inference verification and AI-safety audit tools.